Banco de programación y debug remoto para STM32, ESP32 y Arduino

BeagleBoard-xM · OpenOCD (SWD) · USB DFU · esptool · avrdude · acceso por Tailscale

Documento de trabajo — actualizado al 30/09/2026

Estado: la aplicación está instalada y funcionando en la BeagleBoard-xM, con acceso remoto por Tailscale; el STM32H503 se flashea y se depura con GDB desde la PC, a través de la placa. Se sumó una pestaña Depurar para depurar desde la misma web (probada en simulación; falta GDB 12 en la placa). Se agregó soporte para ESP32/ESP8266 y Arduino (AVR), con cada binario vinculado a su placa; está probado en simulación y falta probarlo con placas reales.

Resumen. Una BeagleBoard-xM sin pantalla ni teclado, ubicada en un banco de laboratorio, funciona como programador y depurador remoto de microcontroladores STM32, ESP32 y Arduino. Desde el navegador de una PC en otro lugar, conectada por una VPN Tailscale, se sube el firmware, se programa el chip por ST-Link (SWD), por el bootloader USB DFU de ST o por puerto serie (esptool para ESP32, avrdude para Arduino), se depura con GDB, se ve la salida en vivo y un monitor serie, y se controla la placa (apagar / reiniciar).

Contenido

Resumen en siete pasos
1. La idea
2. Arquitectura del sistema (diagrama de bloques)
3. Arquitectura de software (diagrama de bloques)
4. Métodos de programación: STM32, ESP32 y Arduino
5. Seguridad
6. Configuración
7. Desarrollo y pruebas sin hardware
8. Puesta en marcha de la BeagleBoard-xM (bitácora)
9. Firmware de prueba para el STM32H503
10. Lecciones aprendidas
11. Próximos pasos

Anexo A. Estructura del repositorio
Anexo B. Comandos útiles

Resumen en siete pasos

Una versión corta de todo el proyecto (sirve también como guion para presentarlo).

Paso	Qué se hizo
0 · La idea	Programar y depurar un STM32 desde cualquier lugar: el micro queda en el banco de laboratorio conectado a una BeagleBoard y se controla desde el navegador de la PC.
1 · Hardware	BeagleBoard-xM sólo con alimentación y red, sin pantalla ni teclado. Un ST-LINK/V2 en su USB y cuatro cables al STM32H503: SWDIO, SWCLK, 3,3 V y GND.
2 · Preparar la placa	La placa tenía Ubuntu 16.04 (2017). Se agrandó la partición y se compiló en la placa lo que faltaba: Python 3.9, dfu-util y el OpenOCD de ST.
3 · Acceso remoto	Tailscale: red privada cifrada. La placa no queda expuesta a internet; sólo los equipos de la cuenta pueden entrar.
4 · Aplicación web	Corre en la placa: subir firmware, elegir chip y método, Flashear, Verificar, Borrar, Reset, con el log de OpenOCD en vivo; monitor serie, historial y apagado seguro.
5 · Firmware de prueba	Tres programas cortos para el H503 con LEDs en PA0–PA7: parpadeo, luz que va y viene y contador binario.
6 · El problema real	Desde la PC conectaba y desde la placa no: faltaba la masa común (GND) entre el ST-Link y la placa del micro. Con ese cable, conectó.
7 · Resultado	Flashear desde el navegador, iniciar el servidor GDB y, desde la PC, detener el micro, leer una variable, cambiarla y ver a los LEDs responder.

> El chip se elige desde un archivo de configuración, así que el banco sirve para cualquier STM32 y, desde la última versión, también para ESP32/ESP8266 y Arduino (28 placas en el catálogo). Todo el código, los scripts y este documento están versionados en GitHub (repositorio privado ElianUlrich/STM32-bench).

1. La idea

El problema. Los microcontroladores con los que se trabaja están conectados en un banco de pruebas de laboratorio, pero quien los programa y depura está en otro lugar. Sin una herramienta remota, cada cambio de firmware exige estar físicamente frente al banco.

La propuesta. Convertir una BeagleBoard-xM en un *banco de programación y debug remoto*: la placa queda conectada al programador (ST-Link) y al STM32, sólo con alimentación de 5 V y cable de red, y ofrece una aplicación web accesible únicamente a través de una VPN (Tailscale). Todo lo que se haría con el programador enchufado a la PC se hace desde el navegador.

Objetivos

- **Programar** cualquier STM32: flashear (program + verify + reset), sólo verificar, borrar la flash completa, reset, halt y run.
- **Dos métodos de conexión** seleccionables por chip o por operación: ST-Link por SWD con OpenOCD (programación y debug) y USB DFU con dfu-util (sólo programación, sin debug).
- **Otras familias**: ESP32/ESP8266 por puerto serie con esptool y Arduino (AVR) con avrdude, por su bootloader o con un programador ISP.
- **No grabar el micro equivocado**: cada binario se sube indicando para qué placa es; la app revisa el contenido y, antes de escribir, confirma el chip conectado.
- **Depurar remotamente** con GDB: arm-none-eabi-gdb, VS Code (Cortex-Debug) o STM32CubeIDE.
- **Gestionar el firmware**: subir .elf/.hex/.bin arrastrando al navegador o sincronizar una carpeta con Syncthing.
- **Ver en vivo** la salida de OpenOCD/dfu-util, un historial de operaciones y un monitor serie.
- **Controlar la placa**: estado del sistema, apagar y reiniciar con confirmación.
- **Soporte configurable**: agregar un chip nuevo es agregar una entrada a un archivo YAML.
- **Seguridad**: sin exposición a internet, login, y ningún comando armado con texto del usuario.

Requisitos y restricciones

Aspecto	Detalle
Placa	BeagleBoard-xM: TI DM3730 (Cortex-A8, ARMv7, armhf), 512 MB de RAM, arranque desde microSD, 4 puertos USB host, Ethernet. Sin pantalla, teclado ni mouse (headless).
Programador	ST-Link V2 / V2-1 / V3, integrado en Nucleo/Discovery o externo, por USB.
Chips actuales	STM32H503CBT6 (Cortex-M33, serie H5) y STM32H723VGT6 (Cortex-M7, serie H7); además ESP32 (todas las variantes) y Arduino Uno, Nano, Mega, Leonardo, etc.
Red	La PC y la placa se conectan por Tailscale. No se abren puertos en el router.
Liviandad	Pocos recursos: backend Python (FastAPI), frontend sin compilación ni Node.
Operación	Servicios con systemd; arranque automático; usuario sin privilegios.

2. Arquitectura del sistema

La figura 1 muestra los bloques del sistema completo. A la izquierda, la PC del usuario; en el medio, el túnel cifrado de Tailscale; a la derecha, la BeagleBoard-xM con los procesos que corren en ella y el hardware de programación conectado a sus puertos USB.

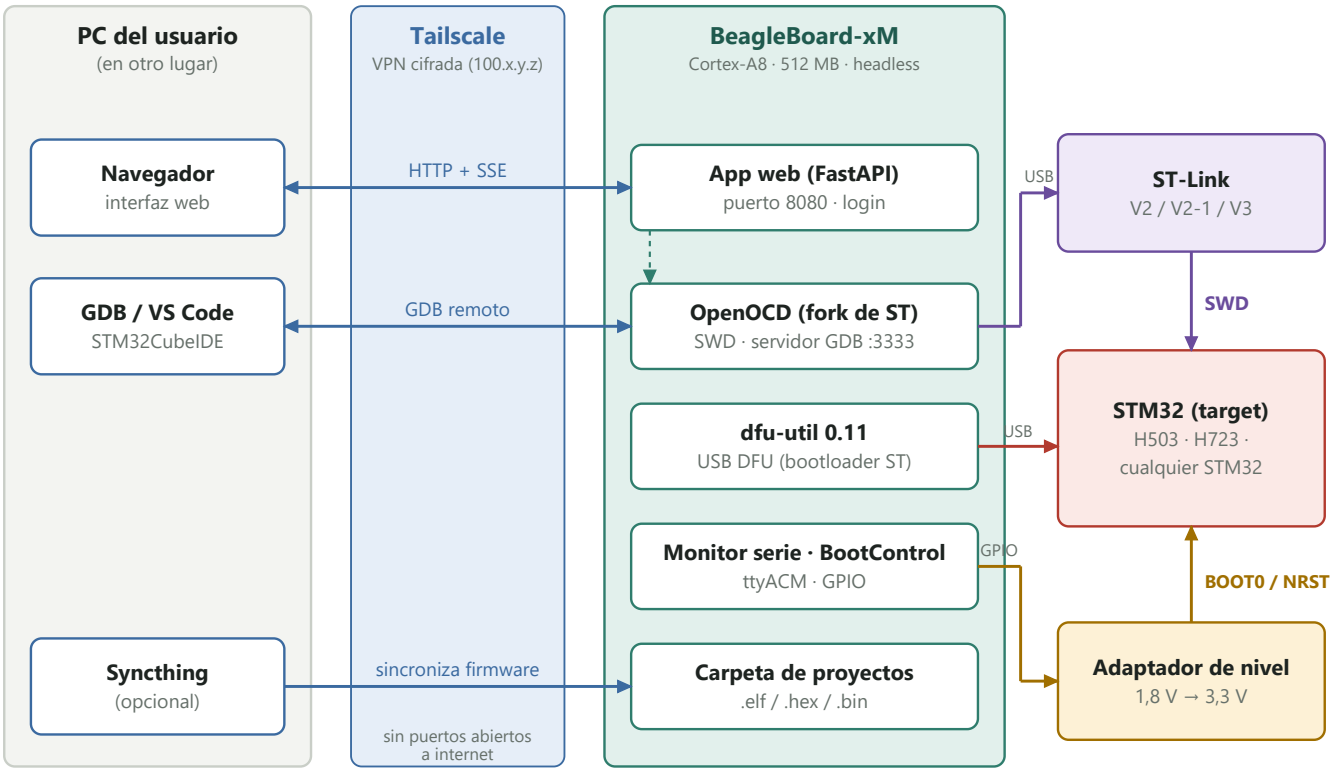


Figura 1 — Diagrama de bloques del sistema. Las flechas azules pasan por la VPN; las de la derecha son conexiones físicas (USB, SWD y GPIO).

Bloques

- **PC del usuario**: navegador para la interfaz web y cliente GDB (arm-none-eabi-gdb, VS Code o CubeIDE). Opcionalmente, Syncthing para sincronizar la carpeta de compilación.
- **Tailscale**: VPN de malla (WireGuard). La placa recibe una IP 100.x.y.z accesible sólo desde los equipos de la cuenta; no hay que abrir puertos en el router.
- **App web**: servicio systemd en el puerto 8080. Orquesta todo lo demás.
- **OpenOCD** (fork oficial de ST, el de STM32CubeIDE): programa por SWD y, en modo servidor, expone el puerto GDB 3333. Se compila en la placa porque el de los repositorios es viejo para H5/H7.
- **dfu-util**: programa por USB a través del bootloader de sistema del STM32 (VID:PID 0483:df11).
- **esptool y avrdude**: programan ESP32/ESP8266 y Arduino por el puerto USB-serie de cada placa (/dev/ttyUSB* o /dev/ttyACM*). No aparecen en la figura para no recargarla: se conectan igual que el monitor serie.
- **ST-Link**: conectado por USB a la placa y por SWD (SWDIO, SWCLK, GND, NRST) al STM32.
- **USB del propio STM32** (PA11/PA12): usado para DFU. En una Nucleo es un conector distinto al del ST-Link, así que pueden estar los dos enchufados.
- **Adaptador de nivel** (opcional, para BootControl por GPIO): los GPIO de la BeagleBoard-xM son de 1,8 V y BOOT0/NRST del STM32 son de 3,3 V; no se pueden conectar directo.

3. Arquitectura de software

Dentro de la placa, la aplicación está organizada en capas (figura 2). Cada pedido del navegador atraviesa el middleware de seguridad, llega a la API y se valida en el servicio central (*Bench*) antes de construir cualquier comando. Los procesos externos se lanzan siempre con una lista de argumentos, nunca a través de una shell.

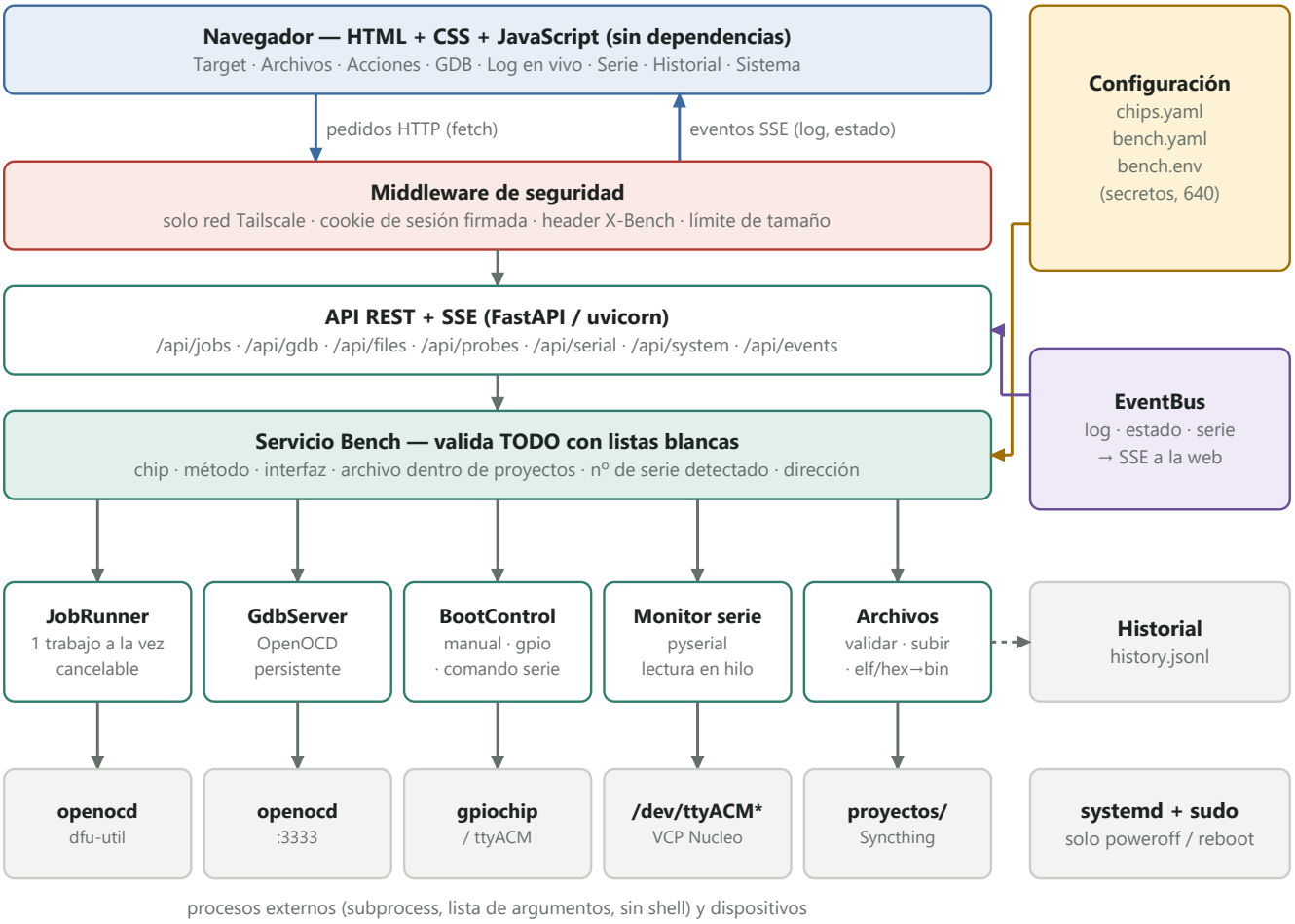


Figura 2 — Bloques de software en la BeagleBoard-xM.

Decisiones técnicas

Decisión	Motivo
FastAPI + uvicorn (sin extras)	Asincronismo nativo para SSE, procesos largos y el monitor serie. ~40–60 MB de RAM. Sin extensiones que compilar en armhf.
Server-Sent Events (SSE)	El log es de una sola vía; SSE no necesita librerías y el navegador reconecta solo. El envío por serie usa POST.
Frontend HTML + CSS + JS sin dependencias	Sin Node ni compilación; se sirve desde el mismo backend. Dimensionado para 1366×768.
Contraseña con scrypt (hashlib)	KDF recomendado por OWASP, en la biblioteca estándar: no hay que compilar bcrypt/argon2 en armhf.
Un trabajo a la vez (409 si está ocupado)	Nunca se pisan dos sesiones del programador; encolar flasheos a ciegas es más riesgoso.
Procesos con Popen + hilo lector	Mismo código en Linux (placa) y Windows (desarrollo), salida línea a línea al log en vivo.
Modo simulación (BENCH_MOCK=1)	Simuladores de OpenOCD, dfu-util, ST-Link, DFU, serie y sistema para desarrollar y probar sin hardware.

4. Métodos de programación: STM32, ESP32 y Arduino

STM32: ST-Link (SWD) y USB DFU

Cada chip declara en **chips.yaml** qué métodos soporta y sus parámetros. La interfaz sólo habilita los métodos disponibles y, con DFU, deshabilita lo que el bootloader no puede hacer, explicando el motivo.

Acción	ST-Link (SWD) — OpenOCD	USB DFU — dfu-util
Flashear	program … verify reset exit	-s 0x08000000:leave -D firmware.bin
Verificar	halt · verify_image · resume	leer con -U y comparar
Borrar flash	flash erase_sector <banco> 0 last	-s 0x08000000:mass-erase:force
Reset / Halt / Run	sí	no (sin acceso al núcleo)
Servidor GDB	sí (puerto 3333)	no
.elf / .hex	directo	se convierten a .bin en la placa

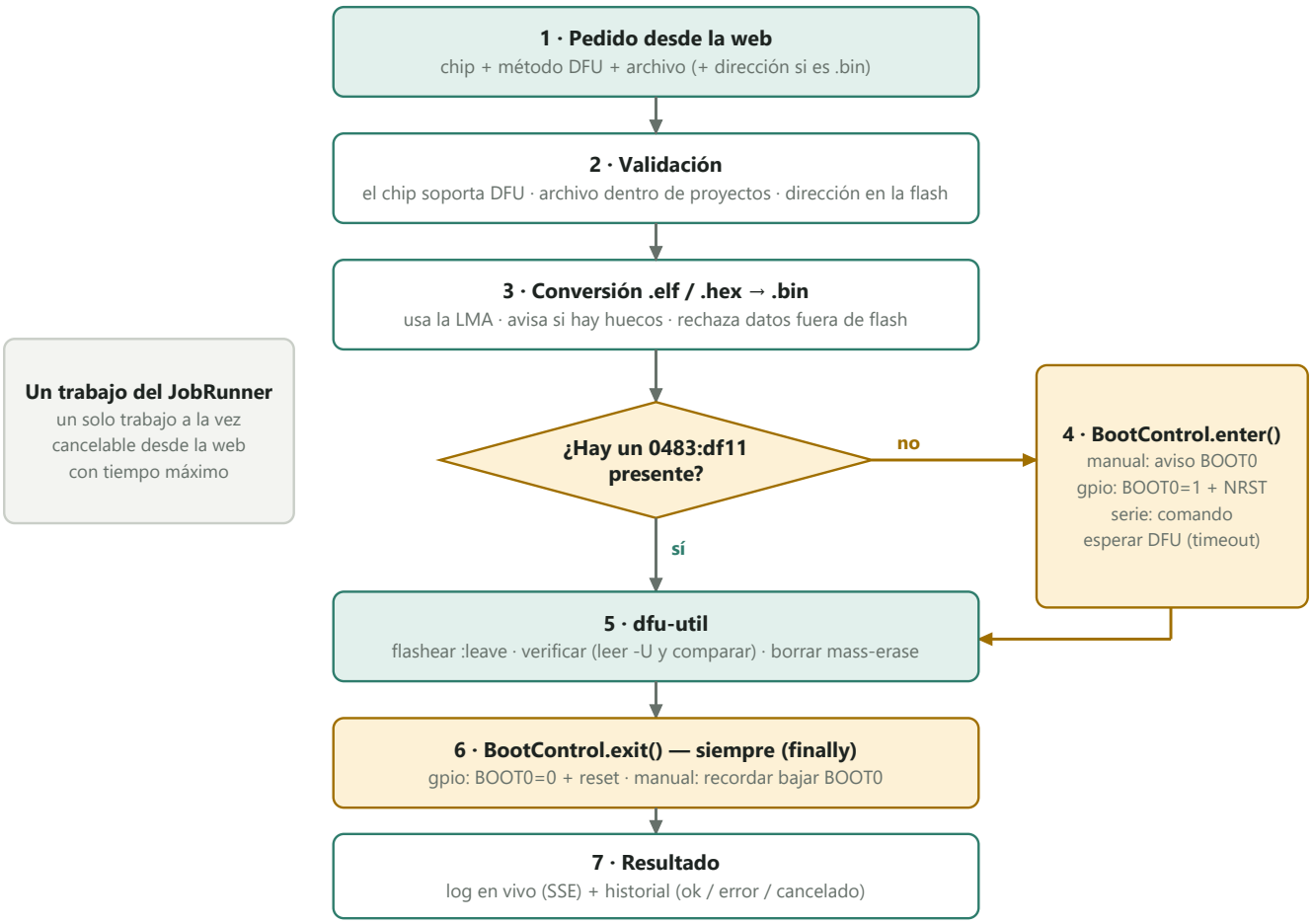


Figura 3 — Flujo de una operación por USB DFU.

Ejemplo de comando construido por la app para flashear un STM32H723 (lista de argumentos; los valores entre llaves son literales en Tcl):

```
openocd -s /usr/local/share/openocd/scripts
        -f interface/stlink-dap.cfg  -c "transport select dapdirect_swd"
        -c "adapter serial 066DFF485550755187121832"
        -f target/stm32h7x.cfg       -c "adapter speed 4000"
        -c "program {/srv/stm32-bench/proyectos/demo/Debug/demo.elf} verify reset exit"
```

> **Hallazgo al revisar el fork de ST (tag openocd-cubeide-v2.2.0):** la CPU del STM32H5 está en el AP 1 del DAP, por lo que el modo HLA (`interface/stlink.cfg`) no sirve; hace falta `stlink-dap.cfg` con `transport select dapdirect_swd`, que ese archivo no elige solo. El ST-Link se selecciona con `adapter serial`.

Backends de BootControl (cómo se entra al bootloader)

Backend	Cómo funciona	Requisitos
manual	La web muestra "Poné BOOT0 en alto y reseteá la placa" con cuenta regresiva y sigue sola cuando aparece el dispositivo DFU.	Una persona en el banco.
gpio	La placa maneja BOOT0 y NRST con libgpiod (/dev/gpiochipN, nunca sysfs): BOOT0=1, pulso de NRST, espera DFU; al final BOOT0=0 y reset.	Adaptador 1,8 V → 3,3 V (NPN en NRST, traductor no inversor en BOOT0).
serial_command	Envía un comando configurable (p. ej. "BOOTLOADER\n") por el puerto serie al firmware, que salta al bootloader de sistema.	Soporte en el firmware (dirección de System Memory según AN2606).

ESP32 / ESP8266 (esptool) y Arduino / AVR (avrdude)

Se agregaron dos métodos más, que usan el puerto USB-serie de cada placa. Cada placa del catálogo declara su método y sus parámetros; la interfaz agrupa las placas por familia, pide el puerto serie y deshabilita (explicando el motivo) lo que el método no puede hacer.

Acción	ESP32 / ESP8266 — esptool	Arduino — avrdude
Flashear	write_flash en 0x10000 (app) o 0x0 (imagen combinada), modo/frecuencia/tamaño "keep"	-U flash:w:archivo.hex:i (con -D en el Mega)
Verificar	verify_flash (compara el hash)	-U flash:v:archivo.hex:i
Borrar	erase_flash	sólo con programador ISP (-e); el bootloader no borra
Reset	reset por DTR/RTS	pulso de DTR (o toque de 1200 baudios en 32U4)
Identificar	flash_id con --chip auto	lee la firma del chip
Halt / Run / GDB	no (haría falta JTAG)	no

- **Catálogo**: 28 placas en chips.yaml: 7 STM32, 11 Espressif (ESP8266, ESP32, S2, S3, C2, C3, C5, C6, C61, H2, P4) y 10 AVR (Uno, Nano con bootloader nuevo y viejo, Pro Mini, Duemilanove, Mega 2560 y 1280, Leonardo, Micro y un ATmega328P genérico con USBasp).
- **Qué archivo se sube**: en Arduino IDE, *Exportar binario compilado*. Para ESP32, el .ino.bin (sólo la aplicación) o el .ino.merged.bin (bootloader + particiones + app); la app reconoce cuál es y elige la dirección. Para AVR, el .ino.hex sin bootloader.
- **Leonardo y Micro** (ATmega32U4): la app abre el puerto a 1200 baudios para que el chip salte al bootloader, espera el puerto nuevo que aparece y graba ahí.
- **Monitor serie**: si tiene abierto el mismo puerto, se cierra durante la grabación y se vuelve a abrir al terminar.
- **Instalación**: avrdude desde apt; esptool 4.12 con pip --no-deps, porque su dependencia cryptography (sólo la usa espsecure) no tiene wheel para la placa.

Vínculo binario ↔ placa

Con placas de tres familias en el mismo banco, grabar un binario en el micro equivocado es fácil. Por eso cada archivo se sube indicando para qué placa es, y la protección tiene tres capas (figura 4): se revisa el contenido del archivo, se exige que la placa elegida sea la del archivo y, antes de escribir, la herramienta confirma el chip conectado. En STM32 esta última capa es nueva: OpenOCD lee el registro DBGMCU_IDCODE y aborta con "CHIP EQUIVOCADO" si no coincide con el esperado.

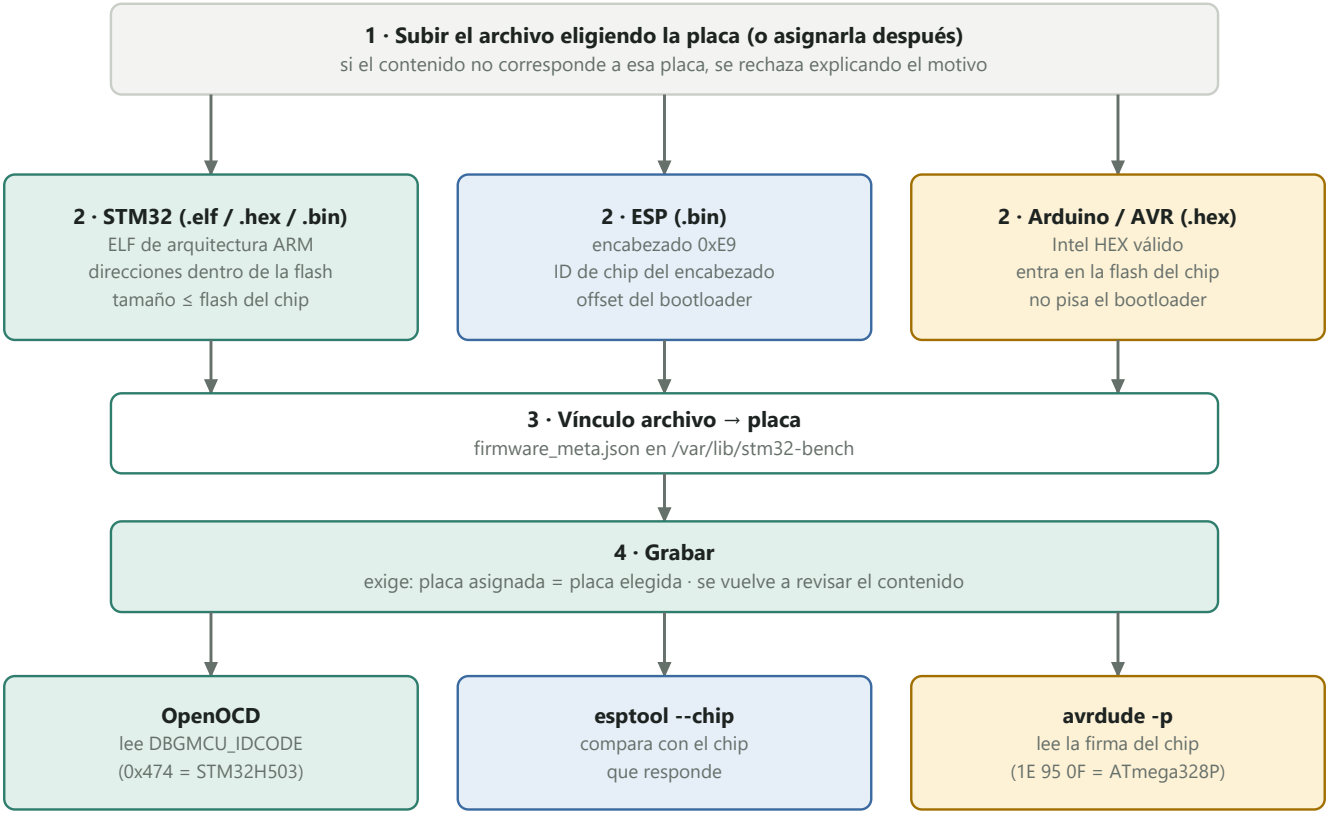


Figura 4 — Las tres capas que evitan grabar un binario en la placa equivocada.

Limitación. Un .hex de AVR no dice para qué chip fue compilado: un sketch de Uno y uno de Nano (el mismo ATmega328P) son indistinguibles. Ahí la protección es la asignación al subirlo más la firma que verifica avrdude. En el ESP8266 el encabezado tampoco trae el ID de chip.

Depurar en la web (pestaña Depurar)

Con el servidor GDB ya se podía depurar desde la PC, pero hacía falta instalar GDB o un IDE y escribir comandos. La pestaña Depurar hace lo mismo desde el navegador: la app corre GDB en la placa en **modo MI** (la interfaz de GDB pensada para programas, la misma que usan VS Code y CubeIDE), lo conecta al OpenOCD local y muestra el resultado (figura 5).

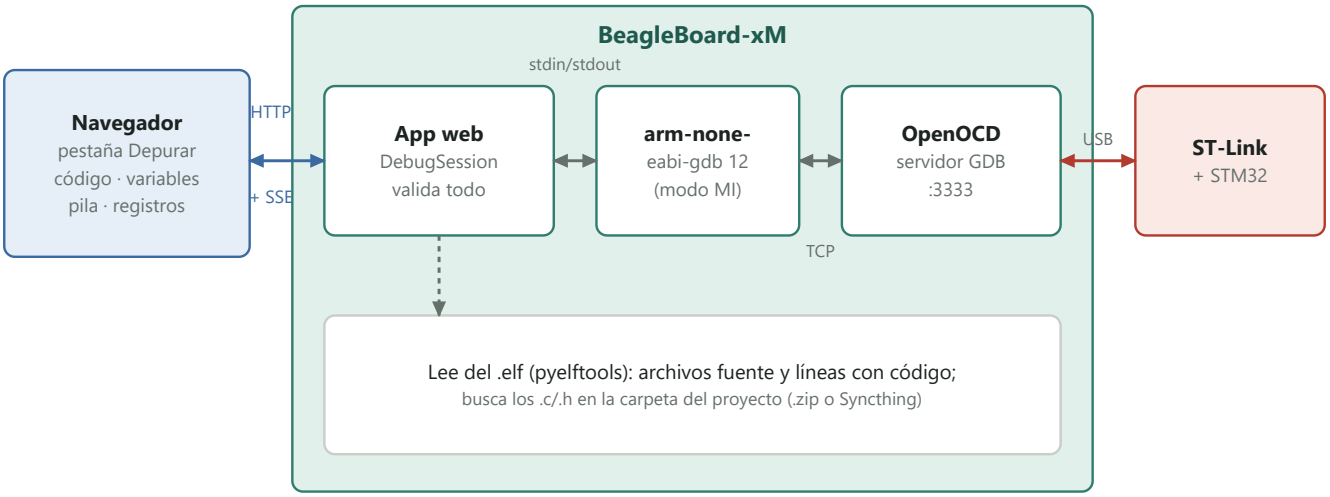
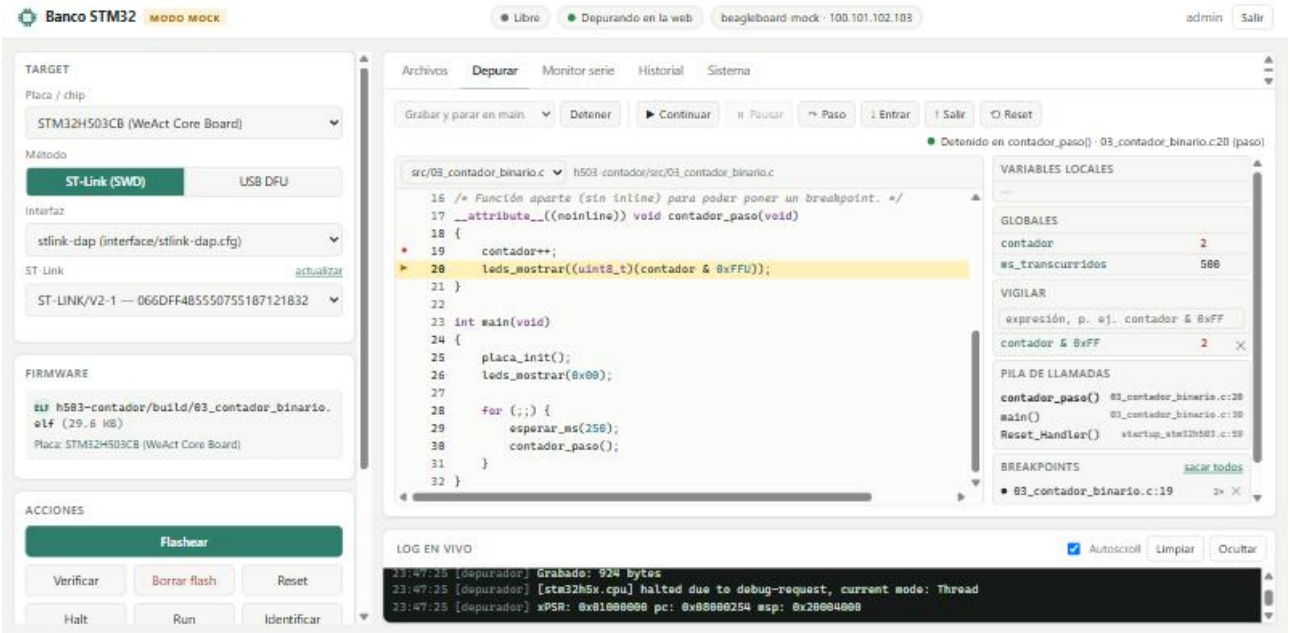


Figura 5 — Depuración en la web: la app maneja GDB en la placa.



Captura — pestaña Depurar (modo simulación): detenido en contador_paso() después de un breakpoint en la línea 19 y un paso. A la derecha, globales (en rojo las que cambiaron), una expresión vigilada, la pila de llamadas y los breakpoints.

Control	Qué hace	Comando MI
Iniciar	Grabar y parar en main / reiniciar y parar en main / conectarse donde esté. Verifica el ID del chip antes de grabar.	-target-select, -target-download
Continuar (F5) / Pausar (F6)	Corre hasta un breakpoint / frena donde esté	-exec-continue / -exec-interrupt
Paso (F10) / Entrar (F11) / Salir (Shift+F11)	Una línea sin entrar a funciones / entrando / hasta volver a quien llamó	-exec-next / -exec-step / -exec-finish
Clic en un número de línea	Pone o saca un breakpoint (de hardware en la flash)	-break-insert archivo:línea

Control	Qué hace	Comando MI
Clic en un valor	Cambia una variable (sólo números, caracteres y true/false)	-data-evaluate-expression var=valor
Reset	Reinicia el micro y queda detenido	monitor reset halt

- **Fuentes**: el .elf guarda las rutas de la PC donde se compiló (C:/Users/.../src/placa.c). La app lee la tabla de líneas DWARF con pyelftools y busca cada archivo en la carpeta del proyecto: relativo a la carpeta del .elf y sus padres, o por nombre. Para eso se sube el proyecto en .zip (se extraen sólo firmware y fuentes, con protección contra rutas maliciosas y bombas zip) o se sincroniza con Syncthing.
- **Seguridad**: el navegador nunca le manda texto a GDB. Pide "el archivo número N del .elf", "breakpoint en la línea L", "paso"; la app arma el comando. Las expresiones a vigilar no pueden usar variables de GDB ($...) salvo registros, porque algunas ejecutan cosas en la placa.
- **Un solo cliente**: OpenOCD acepta una sola conexión GDB, así que mientras se depura en la web no se puede conectar un GDB externo. Flashear desde Acciones termina la sesión.
- **GDB 12**: el GDB de Ubuntu 16.04 (7.11) no entiende DWARF 5, el formato del GCC de CubeIDE. scripts/build_gdb.sh compila GDB 12 sólo para ARM en la placa (varias horas, con swap temporal).
- **Pruebas**: un simulador (scripts/mock_gdb.py) imita a GDB ejecutando el contador binario con sus líneas reales; con él se probó la pestaña completa y los tests cubren sesiones enteras.

5. Seguridad

La aplicación puede ejecutar comandos en la placa, así que la seguridad se diseñó en capas:

- **Red**: acceso sólo por Tailscale; no se abren puertos en el router. Además la app rechaza conexiones que no vengan de las redes de Tailscale (100.64.0.0/10, fd7a:115c:a1e0::/48) o de localhost.
- **Autenticación**: usuario y contraseña; hash scrypt (o PBKDF2-HMAC-SHA256 con 600.000 iteraciones si el Python no trae scrypt, como en la placa); cookie de sesión firmada, HttpOnly, SameSite=Strict, con vencimiento; límite de intentos fallidos por IP. Sin credenciales en el código: van en /etc/stm32-bench/bench.env (permisos 640).
- **CSRF**: toda petición que modifica algo exige el header X-Bench (un sitio ajeno no puede agregarlo sin CORS, que no está habilitado).
- **Inyección de comandos**: subprocess con lista de argumentos y sin shell; chips, interfaces y backends sólo desde los YAML; rutas resueltas dentro del directorio de proyectos (sin ".." ni symlinks hacia afuera); lista blanca de caracteres en los nombres, porque los argumentos -c de OpenOCD son código Tcl y una ruta con [ ] { } $ podría inyectar Tcl.
- **Privilegios**: el servicio corre como usuario stm32bench (grupos plugdev, dialout, gpio). sudo sólo permite poweroff y reboot sin argumentos, validado con visudo -c.
- **GDB**: el puerto 3333 no tiene autenticación, por eso escucha sólo en la IP de Tailscale y OpenOCD se lanza con telnet y Tcl deshabilitados. Se recomiendan ACL de Tailscale.
- **Cabeceras HTTP**: Content-Security-Policy estricta, X-Frame-Options DENY, nosniff, no-referrer.

6. Configuración

Los archivos viven en /etc/stm32-bench/ (en el repositorio, en config/). Agregar un chip es agregar una entrada:

```
- id: stm32h723vg
  name: STM32H723VGT6
  family: H7 (Cortex-M7)
  flash: { base: 0x08000000, size_kb: 1024, sector_kb: 128 }
  default_method: swd
  methods:
    swd:
      target_cfg: target/stm32h7x.cfg      # relativo a los scripts de OpenOCD
      interface: stlink-dap                # definida en bench.yaml
      adapter_speed_khz: 4000
      flash_banks: [0]                     # bancos a borrar
    dfu:
      vid_pid: "0483:df11"
      alt: 0                               # "Internal Flash"
```

Archivo	Contenido
chips.yaml	Catálogo de 28 placas: STM32 (H503, H723 y ejemplos F1, F4, L4, G0, G4, cada uno con su IDCODE), ESP32/ESP8266 y Arduino. Al arrancar se verifica que los .cfg existan en el OpenOCD instalado.
bench.yaml	Binarios y timeouts de OpenOCD, dfu-util, esptool y avrdude, interfaces (stlink = HLA, stlink-dap = dapdirect), puerto y bind del GDB, tamaño máximo de subida, baudrates, redes permitidas, BootControl.
bench.env	Usuario, hash de la contraseña, clave de sesión y rutas. Lo genera install.sh.

7. Desarrollo y pruebas sin hardware

Para desarrollar en la PC (Windows) sin la placa, el **modo simulación** (BENCH_MOCK=1) reemplaza OpenOCD, dfu-util, esptool y avrdude por simuladores que se ejecutan como procesos reales, así el camino de ejecución y el log en vivo son los mismos que en la placa. También simula dos ST-Link, la aparición del dispositivo DFU al entrar al bootloader, un puerto serie y los datos del sistema. Se crean proyectos de ejemplo, incluidos casos de error (hueco entre segmentos, datos en flash externa, verificación fallida).

```
powershell -ExecutionPolicy Bypass -File scripts\dev_run.ps1   # http://127.0.0.1:8080
```

Pruebas automáticas

228 tests con pytest, todos pasando (uno se omite en Windows). Cubren:

- Validación de archivos: nombres, extensiones, tamaño, contenido (ELF ARM de 32 bits, Intel HEX), path traversal y symlinks.
- Construcción de comandos de OpenOCD y dfu-util: orden de argumentos, direcciones, bloqueo de inyección Tcl, parseo de dfu-util -l.
- Carga y validación de chips.yaml y bench.yaml; detección de .cfg faltantes.
- Selección de método según el chip y capacidades de cada método.
- Conversión .elf/.hex → .bin: uso de la LMA, huecos, datos fuera de flash, solapamientos.
- BootControl: secuencia de GPIO con un driver falso, manual, comando serie, timeout, cancelación y flujo DFU completo con el simulador.
- API: login, redes permitidas, CSRF, trabajos concurrentes (409), bloqueo por servidor GDB activo, monitor serie, apagado.
- Autenticación: hash scrypt y respaldo PBKDF2 cuando hashlib.scrypt no existe.
- ESP32 y Arduino: análisis de .bin (ID de chip, imagen combinada, offset del bootloader) y de .hex (tamaño, bootloader), comandos de esptool y avrdude, puertos inválidos, toque de 1200 baudios y liberación del monitor serie.
- Depurador: protocolo GDB/MI, validación de expresiones, fuentes del .elf (DWARF 3 y 5, rutas de Windows), proyectos .zip y sesiones completas con el GDB simulado.
- Vínculo binario ↔ placa: subida sin placa, placa incompatible, reasignación, grabación rechazada en otra placa, firma AVR distinta y chip STM32 equivocado (IDCODE).

Pruebas en el navegador

Con la app en modo simulación se recorrieron los flujos principales: flasheo por SWD con log en vivo, flasheo por DFU con el diálogo “Poné BOOT0 en alto…” (que se cierra solo al aparecer el dispositivo) y el aviso de segmentos discontinuos, servidor GDB (muestra IP:3333) y el pedido de detenerlo al flashear, monitor serie, historial y estado del sistema, a 1366×678.

Problemas encontrados y corregidos en esas pruebas:

- El diálogo de confirmación dependía del evento *close* del elemento <dialog>, que el navegador no emite si la pestaña no se está dibujando: ahora se resuelve desde los botones.
- El navegador usaba CSS/JS viejos de la caché: los archivos estáticos ahora se sirven con Cache-Control: no-cache (revalidación con ETag).
- La lista de archivos repetía el nombre del proyecto como subcarpeta.

Con ESP32 y Arduino se probó: grabar un Uno desde la web, que al elegir un archivo se seleccione sola su placa, que un binario de ESP32-S3 no se pueda grabar en un ESP32 (el botón queda deshabilitado con el

motivo) y grabar una imagen combinada de ESP32 en 0x0 con el hash verificado. Se corrigieron dos errores: la validación al subir usaba la extensión del archivo temporal (.part) y el puerto serie elegido para una placa se arrastraba a otra (ahora se recuerda por placa).

8. Puesta en marcha de la BeagleBoard-xM (bitácora)

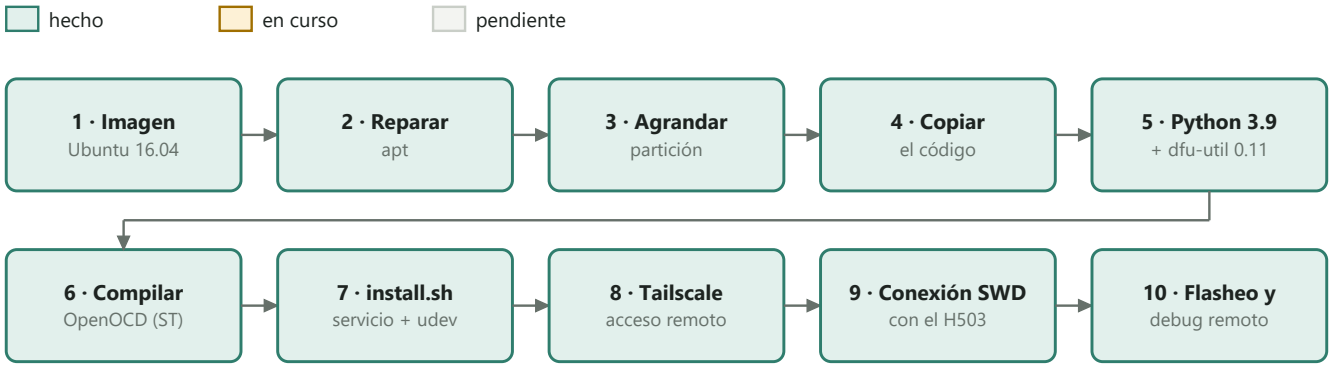


Figura 6 — Pasos de la puesta en marcha y su estado al 30/09/2026.

8.1 Estado inicial encontrado

Elemento	Encontrado
Sistema	Ubuntu 16.04 LTS, imagen de consola de rcn-ee (Robert C. Nelson) del 23/01/2017.
Kernel	4.9.5-armv7-x4 (armv7l).
Memoria	483 MB utilizables, sin swap.
Almacenamiento	microSD de 3,7 GB, pero la partición raíz ocupaba sólo 1,6 GB (474 MB libres).
Python	3.5.2 (la app necesita 3.9 o superior).
Red	eth0 192.168.0.27/24 (LAN) y usb0 192.168.7.2 (red por USB de la imagen).
apt	Instalación de paquetes interrumpida y un repositorio https sin soporte instalado.

8.2 Decisión: mantener Ubuntu 16.04

Se evaluó instalar un sistema más nuevo, pero la página oficial de imágenes de BeagleBoard.org ya no publica imágenes para la xM; una Debian nueva habría que armarla a partir del sistema base de rcn-ee más kernel y bootloader. Actualizar en el lugar (16.04 → 18.04 → 20.04 → 22.04) es lento y riesgoso con un kernel no estándar. Como los repositorios de 16.04 siguen respondiendo, se decidió conservar el sistema y agregar un Python 3.9 compilado aparte (la última versión que compila con el OpenSSL 1.0.2 de 16.04). El acceso sólo por Tailscale compensa la falta de actualizaciones de seguridad.

8.3 Acceso y reparación de apt

Con monitor y teclado se ingresó con el usuario por defecto de la imagen (indicado en la pantalla de login) y luego se trabajó por SSH desde la PC, para copiar y pegar comandos. Se recomienda cambiar la contraseña de fábrica con `passwd`.

```
ssh ubuntu@192.168.0.27
sudo dpkg --configure -a
sudo apt install -y apt-transport-https ca-certificates
sudo apt update          # OK: ports.ubuntu.com (xenial) y repos.rcn-ee.com responden
```

8.4 Agrandar la partición a toda la microSD

```
sudo apt install -y cloud-guest-utils tmux
sudo growpart /dev/mmcblk0 2        # CHANGED: partition=2 ... new: size=7690207
sudo resize2fs /dev/mmcblk0p2       # redimensionado en línea, sin reiniciar
df -h /                             # 3,6 GB de tamaño, 2,4 GB libres
```

8.5 Copia del código a la placa

El repositorio todavía no está publicado; se empaquetó en la PC y se copió por SSH (219 KB).

```
# En la PC (PowerShell)
tar --exclude=.venv --exclude=dev-data --exclude=.git -czf $env:USERPROFILE\Desktop\stm32-bench.tgz
    -C $env:USERPROFILE\Desktop\Debugger .
scp $env:USERPROFILE\Desktop\stm32-bench.tgz ubuntu@192.168.0.27:~
# En la placa
mkdir -p ~/stm32-bench && tar -xzf ~/stm32-bench.tgz -C ~/stm32-bench && cd ~/stm32-bench
```

8.6 Preparación del sistema (completada)

Se agregó el script `scripts/prepare_ubuntu1604.sh`, que se ejecuta dentro de tmux (si se corta el SSH, el trabajo sigue; se retoma con `tmux attach -t prep`):

- Instala las dependencias de compilación y crea un swap temporal de 512 MB.
- Descarga Python 3.9.25, verifica su SHA-256, lo compila e instala en /opt/python3.9 sin tocar el Python del sistema (1 a 2 horas en la xM).
- Compila dfu-util 0.11 en /usr/local (el de 16.04 es 0.8 y no tiene mass-erase), verificando su MD5.
- Borra los archivos de compilación para ahorrar espacio.

```
tmux new -s prep
sudo bash scripts/prepare_ubuntu1604.sh
```

Resultado: terminó en unos 72 minutos (20:02 → 21:14, hora de la placa). Quedaron Python 3.9.25 con OpenSSL 1.0.2g (todos los módulos necesarios cargan) y dfu-util 0.11. Durante la compilación sólo aparecieron advertencias del compilador (variables sin usar, strict-aliasing), sin errores; las advertencias de pip por ejecutarse como root son esperables porque la app usa su propio entorno virtual.

8.7 Tailscale (hecho en paralelo a la compilación)

Desde otra sesión SSH, mientras compilaba Python. La imagen no traía `curl` (se agregó al script de preparación). El repositorio de Tailscale todavía publica paquetes para Ubuntu 16.04 armhf (versión 1.102.4).

```
sudo apt install -y curl
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --hostname=banco-stm32     # se autoriza abriendo el enlace en la PC
```

En la consola de Tailscale (Machines → banco-stm32 → □ → Disable key expiry) se desactivó el vencimiento de la clave, para que la placa no quede fuera de la red. La placa obtuvo la IP **100.99.235.81** y se verificó el acceso remoto desde la PC con `ssh ubuntu@100.99.235.81`. Desde ahí se usa esa IP (o el nombre banco-stm32) en lugar de la IP de la red local.

Otros ajustes hechos para 16.04: `install.sh` detecta /opt/python3.9, instala pydantic-core sólo como paquete precompilado (se verificó que existe para armv7 y Python 3.9 hasta la versión 2.46.5) y ya no usa piwheels; `build_openocd.sh` clona sólo la versión necesaria y borra el código al terminar. En 16.04 el backend GPIO de BootControl queda pendiente (no hay python3-libgpiod y el kernel 4.9 no tiene la API v2 de GPIO).

8.8 Compilación de OpenOCD

Con `sudo bash scripts/build_openocd.sh` dentro de tmux: dependencias, clon superficial del fork de ST (tag openocd-cubeide-v2.2.0), swap temporal de 1 GB y compilación. **Tardó 35 minutos** y quedó instalado `Open On-Chip Debugger 0.12.0+dev-gc8d973b`. El script terminó antes de la verificación final por un error propio: `openocd --version | head -n 1` con `set -o pipefail`: `head` cierra la tubería, OpenOCD recibe SIGPIPE y el script aborta. Se corrigió (`sed -n 1p` y `grep ... >/dev/null` en lugar de `head` y `grep -q`) y los

.cfg se verificaron a mano.

8.9 Dos problemas detectados antes de instalar

- **Fin de línea de Windows (CRLF).** Algunos archivos editados en la PC quedaron con CRLF y bash falló con `set: pipefail: invalid option`. Se convirtieron en la placa con sed, y desde entonces el paquete para la placa se genera con `git archive` a partir del repositorio (siempre LF, sin archivos temporales).
- **hashlib.scrypt no existe con OpenSSL 1.0.2.** El Python compilado en la placa no trae scrypt, que se usaba para el hash de la contraseña de la web. Se agregó un respaldo con PBKDF2-HMAC-SHA256 de 600.000 iteraciones (recomendación de OWASP) y tests para ese caso.

8.10 Instalación de la aplicación

`sudo bash scripts/install.sh` terminó sin errores en unos 15 minutos:

- python3-libgpiod no existe para 16.04: el script lo avisa y sigue (sólo afecta al backend GPIO).
- pydantic-core 2.46.5 se instaló como paquete precompilado para armv7 (sin compilar Rust); FastAPI 0.128.8, uvicorn 0.39.0 y starlette 0.49.3; PyYAML se compiló en la placa en segundos.
- sudoers validado con visudo: sólo /sbin/poweroff y /sbin/reboot (en 16.04 están en /sbin).
- Servicio systemd stm32-bench activo y habilitado para arrancar con la placa.

La web quedó accesible en **http://100.99.235.81:8080**, sin avisos de archivos .cfg faltantes. La pestaña Sistema mostró: temperatura del procesador de 62 a 65 °C, unos 130 MB de RAM usados de 483 MB, 1,4 GB de 3,6 GB de disco, y las versiones de OpenOCD y dfu-util.

> Buena práctica: la contraseña de la web se ingresa sólo cuando el script la pide. Si por error se escribe como comando, queda en el historial de bash; se borra con `history -c && history -w`.

8.11 Primer reinicio de la placa

Después de apagar y encender, el servicio de la web arrancó solo, pero la placa no era accesible: `ip link show eth0` mostraba `NO-CARRIER` (el cable de red estaba flojo) y Tailscale quedaba en `NoState` por no tener red. Con el cable bien conectado bastó `sudo systemctl restart tailscaled`. También se observó que la xM genera una dirección MAC aleatoria en cada arranque (por ejemplo 2a:62:23:2a:63:0c): queda pendiente fijarla para que el router le asigne siempre la misma IP.

8.12 Primera conexión con el STM32H503 (diagnóstico)

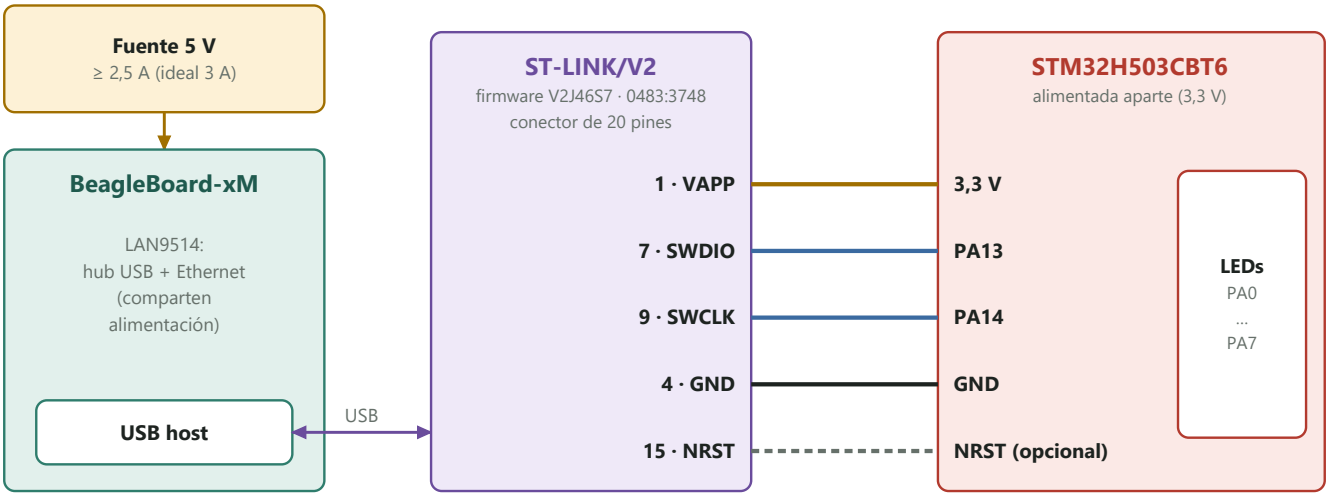


Figura 7 — Cableado del banco: BeagleBoard-xM, ST-LINK/V2 (conector de 20 pines) y placa del STM32H503.

El programador es un **ST-LINK/V2 original** (firmware V2J46S7, compatible con el modo dapdirect que necesita el H5). Su número de serie es binario, como en todos los V2 viejos; la app lo convierte a hexadecimal y OpenOCD lo aceptó con `adapter serial`. La primera conexión falló y se diagnosticó por pasos:

Prueba	Resultado	Conclusión
Reset desde la web (4000 kHz)	ST-Link encontrado y 3,21 V, pero "init mode failed (unable to connect to the target)"	Programador y alimentación del micro bien; falla el SWD
Por SSH a 200 kHz y conectando bajo reset	Mismo error	No es la velocidad
ST-Link en la PC con STM32CubeProgrammer	Conecta: STM32H50x, ID 0x474, 128 KB, AP 1	El cableado SWD y el chip están bien
ST-Link en la PC con el OpenOCD de CubeIDE (con el .cfg de CubeIDE y con el de la placa)	"Examination succeed" en los dos casos	La configuración de OpenOCD no es el problema
De vuelta en la placa, con cable de GND	Avanza más: "Fail reading CTRL/STAT register"	Faltaba la masa común
dmesg en la placa	El Ethernet se cae y vuelve (eth0 link down / link up)	Después se vio que era el cable de red flojo, no la alimentación
Con GND conectado y menos carga en los USB	"Examination succeed" y Reset enviado	Conexión SWD funcionando desde la BeagleBoard

Causa: principalmente la falta de un cable de masa entre el ST-Link y la placa del micro (desde la PC funcionaba porque todo compartía la masa de sus USB). Las caídas del Ethernet que mostraba dmesg resultaron ser un cable de red flojo y no un problema de alimentación. Igual conviene revisar la capacidad de la fuente de 5 V de la placa y, si hace falta, usar un hub USB con fuente propia. La velocidad SWD del H503 se bajó a 1000 kHz en /etc/stm32-bench/chips.yaml, un valor más tolerante con cables largos.

Queda un aviso no bloqueante: al conectar, el script `target/stm32h5x.cfg` de esta versión de OpenOCD intenta configurar DBGMCU (depuración en bajo consumo y watchdogs detenidos) y esa escritura falla. No afecta a los programas de prueba; se corregirá usando el archivo de CubeIDE, que conecta sin ese error.

9. Firmware de prueba para el STM32H503

Para probar el banco se escribieron tres programas mínimos en C, sin HAL, para una placa con el STM32H503CBT6 y LEDs en PA0–PA7. Están en `firmware/h503_leds/` del repositorio, ya compilados (.elf, .hex y .bin).

Programa	Qué hace	Qué prueba del banco
01_blink_todos	Enciende los LEDs de a uno (PA0 → PA7) y después parpadean todos a 1 Hz	Flashear, Verificar y el orden de los LEDs
02_auto_fantastico	Una luz que va y viene de PA0 a PA7	Halt congela la luz, Run la reanuda, Reset reinicia; variable en .data para la conversión DFU
03_contador_binario	Cuenta de 0 a 255 en binario cada 250 ms	Servidor GDB: leer y modificar la variable contador, breakpoints

- **Registros**: direcciones tomadas del encabezado oficial de ST (stm32h503xx.h): RCC en 0x44020C00 (AHB2ENR en +0x8C, bit 0 = GPIOA) y GPIOA en 0x42020000. Sólo se tocan PA0–PA7; PA13 y PA14 quedan libres para el SWD.
- **Reloj**: el código lee el divisor HSIDIV del HSI de reset para calcular los tiempos, así funciona sea cual sea su valor inicial. Base de tiempo de 1 ms con SysTick.
- **Memoria**: flash de 128 KB en 0x08000000 y sólo SRAM1 (16 KB), para no depender de que SRAM2 esté habilitada tras el reset. Arranque y tabla de vectores (16 + 134 entradas) propios, en C.
- **Compilación**: con el GCC 13.3 incluido en STM32CubeIDE, sin advertencias; unos 920 bytes cada programa. La conversión .elf/.hex → .bin de la app coincide byte a byte con la de objcopy.
- **Polaridad**: por defecto el LED enciende con el pin en alto; si es al revés se compila con `make DEFS=-DLED_ACTIVO_ALTO=0`.

Depuración remota con el contador binario

Con el programa flasheado y el servidor GDB iniciado desde la web, en la PC:

```
arm-none-eabi-gdb 03_contador_binario.elf
(gdb) target extended-remote 100.99.235.81:3333
(gdb) monitor halt                # los LEDs se congelan
(gdb) print contador              # coincide con los LEDs
(gdb) set var contador = 0xAA     # LEDs 10101010
(gdb) break contador_paso
(gdb) continue
```

Probado el 30/09/2026. Con el arm-none-eabi-gdb de STM32CubeIDE en la PC (Windows) y el servidor GDB iniciado desde la web, por Tailscale: al conectarse el núcleo quedó detenido en esperar_ms; print contador devolvió 5439 (los LEDs muestran los 8 bits bajos, 0x3F); set var contador cambió los LEDs; continue, Ctrl+C, un breakpoint en contador_paso, delete, quit (el programa sigue corriendo) y la reconexión funcionaron. El script firmware/h503_leds/depurar.bat lanza GDB ya conectado.

Dos tropiezos. Abrir 100.99.235.81:3333 en el navegador no sirve: es el puerto de GDB, no una página. OpenOCD recibe el pedido HTTP como basura (“ignoring character”) y el navegador ocupa la única conexión GDB, así que las siguientes se rechazan. Y si el servidor está detenido, GDB responde “Connection timed out”: hay que tocar Iniciar en la web y esperar “Escuchando”.

10. Lecciones aprendidas

- **Verificar las versiones antes de diseñar sobre ellas.** La placa tenía Ubuntu 16.04 con Python 3.5 y OpenSSL 1.0.2: hubo que compilar Python 3.9 y reemplazar scrypt por PBKDF2.
- **Windows y Linux no escriben igual los fines de línea.** Un script con CRLF no corre en bash. Solución: .gitattributes con eol=lf y paquetes generados con git archive.
- **set -o pipefail tiene trampas.** head o grep -q cierran la tubería antes de tiempo y el comando anterior termina con SIGPIPE, lo que aborta el script.
- **Leer bien la tensión del target no prueba que el SWD funcione.** El pin VAPP sólo mide; SWDIO y SWCLK necesitan una masa común con el programador.
- **Aislar el problema cambiando una sola cosa.** Probar el mismo ST-Link y los mismos cables desde la PC separó en minutos el cableado, la configuración de OpenOCD y la placa.
- **La BeagleBoard-xM tiene particularidades de hardware:** el Ethernet y los USB salen del mismo chip (LAN9514) y comparten alimentación; la MAC es aleatoria en cada arranque; no tiene pila para el reloj.
- **El STM32H5 necesita el modo dapdirect (stlink-dap)**, porque su CPU está en el AP 1; un ST-LINK/V2 original con firmware reciente lo soporta.
- **Una causa a la vez.** Las caídas de Ethernet parecían un problema de alimentación y eran un cable flojo; el SWD se arregló con la masa. Mezclar síntomas lleva a conclusiones equivocadas.
- **Con varias familias en el banco, el archivo tiene que saber para qué placa es.** Los .bin de ESP traen el ID del chip y los STM32 exponen su IDCODE, pero un .hex de AVR no dice nada: la asignación explícita al subir cubre ese hueco.
- **Dependencias en una placa vieja**: esptool trae cryptography, que no compila en armhf con OpenSSL 1.0.2; como sólo la usa espsecure, se instala con --no-deps.
- **El formato de depuración también tiene versiones.** GCC 11+ genera DWARF 5 y un GDB de 2016 no lo lee: antes de diseñar sobre una herramienta del sistema hay que verificar que entienda lo que producen las herramientas de la PC.
- **Un simulador fiel acelera todo.** Simular GDB con el programa real (mismas líneas y funciones) permitió armar y probar la pestaña Depurar completa mientras GDB se compilaba en la placa.
- **Documentar mientras se trabaja** (este documento se genera desde un script y se actualiza en cada paso) evita reconstruir de memoria lo que pasó.

11. Próximos pasos

Paso	Detalle
Otros clientes GDB	Probar VS Code (Cortex-Debug) y STM32CubeIDE contra el servidor GDB del banco; Halt, Run y Reset desde la web con 02_auto_fantastico.
Alimentación	Confirmar que la fuente de la xM sea de 5 V y al menos 2,5 A; si el SWD vuelve a fallar, usar un hub USB con fuente propia para el ST-Link.
Script del H5	Usar el stm32h5x.cfg de CubeIDE para eliminar el error de examine-end (configuración de DBGMCU).
MAC fija	Fijar la dirección MAC de eth0 para que la IP de la red local no cambie.
Despliegue por git	Clonar el repositorio desde GitHub en la placa (con un token de sólo lectura) para actualizar con git pull.
USB DFU	El H503 entra al bootloader pero la BeagleBoard no lo enumera: probar el mismo cable USB-C en la PC (puede ser sólo de carga) y otro puerto de la xM.
Depurar en la web	Terminar de compilar GDB 12 en la placa (build_gdb.sh, en curso), instalar la versión nueva y depurar el contador binario desde la pestaña.

Paso	Detalle
ESP32 y Arduino	Instalar la versión nueva en la placa y probar Identificar en el H503 (IDCODE real), un ESP32 y un Arduino Uno reales.
microSD más grande	Con 8 GB o más: partición de datos y sistema en solo lectura.

Verificaciones pendientes en hardware

- Flashear, verificar, borrar, halt y run en el STM32H503 y, más adelante, en el STM32H723.
- VS Code (Cortex-Debug) y STM32CubeIDE como clientes GDB (arm-none-eabi-gdb ya está probado).
- DFU real: dfu-util -l, :leave, verificación con -U y mass-erase sin -D.
- Monitor serie con un VCP; apagar y reiniciar desde la web.
- Pestaña Depurar con GDB 12 y el OpenOCD real: load, breakpoints de hardware, pausar con mi-async, registros y el tiempo de lectura de un proyecto grande con pyelftools en la xM.
- Dos ST-Link conectados a la vez (selección por número de serie).
- Identificar en STM32: el Tcl read_memory sobre DBGMCU_IDCODE con el OpenOCD de ST.
- esptool 4.12 sin cryptography en la xM; ESP32 y ESP32-S3/C3 (USB-JTAG nativo) reales.
- avrdude 6.2 de Ubuntu 16.04 con Uno, Nano con bootloader viejo, Mega 2560 y Leonardo (1200 baudios).

Anexo A. Estructura del repositorio

```
app/                backend (FastAPI)
  main.py           aplicación, middleware de seguridad, sesiones, estáticos
  api.py            rutas /api/...
  bench.py          servicio central: valida pedidos y arma trabajos SWD/DFU/ESP/AVR
  settings.py       variables de entorno y bench.yaml
  chips.py          chips.yaml (28 placas) y verificación de .cfg
  compat.py         ¿este archivo es para esta placa? (ARM / ESP / AVR)
  firmware_meta.py  vínculo archivo -> placa
  elfinfo.py        fuentes y líneas con código de un .elf (DWARF)
  debugger/         pestaña Depurar: protocolo GDB/MI y sesión de GDB
  files.py          validación, subida, escaneo y borrado de firmware
  convert.py        .elf/.hex -> .bin para DFU
  probe.py          ST-Link por sysfs y dispositivos DFU
  programmers/      capacidades; comandos de OpenOCD, dfu-util, esptool y avrdude;
                    JobRunner, GdbServer
  bootcontrol/      manual, gpio (libgpiod), comando serie, simulado
  serial_monitor.py monitor serie (pyserial)
  system.py         estado del sistema, apagar/reiniciar
  events.py         bus de eventos para SSE
  history.py        historial (JSON Lines)
web/                interfaz (HTML, CSS, JS sin dependencias)
config/             chips.yaml, bench.yaml, bench.example.env
scripts/            build_openocd.sh, build_gdb.sh, prepare_ubuntu1604.sh, install.sh, hash_password.py,
                    dev_run.ps1/.sh, simuladores de OpenOCD, dfu-util, esptool y avrdude
deploy/             unidad systemd, reglas udev, plantilla de sudoers
firmware/h503_leds/ firmware de prueba para STM32H503 (fuentes, Makefile, compilados)
docs/               este documento y su generador
tests/              228 tests (pytest)
```

Anexo B. Comandos útiles

```
ssh ubuntu@100.99.235.81             # entrar a la placa por Tailscale (o ubuntu@banco-stm32)
tmux attach -t prep                  # volver a una sesión de tmux
systemctl status stm32-bench         # estado del servicio
journalctl -u stm32-bench -f         # log del servicio
sudo systemctl restart stm32-bench   # reiniciar tras editar /etc/stm32-bench/*.yaml
lsusb | grep 0483                    # ST-Link / DFU conectados
ls /dev/ttyUSB* /dev/ttyACM*         # puertos serie (ESP32, Arduino, VCP)
df -h / ; free -m                    # espacio y memoria
ip link show eth0                    # NO-CARRIER = cable de red desconectado
sudo systemctl restart tailscaled    # reconectar Tailscale
dmesg | tail -25                     # mensajes del kernel (USB, red)
openocd -f interface/stlink-dap.cfg -c "transport select dapdirect_swd" -f target/stm32h5x.cfg \
        -c "adapter speed 1000" -c init -c "reset halt" -c shutdown     # prueba SWD manual
git archive --format=tar.gz -o stm32-bench.tgz HEAD   # (en la PC) paquete limpio para la placa
```